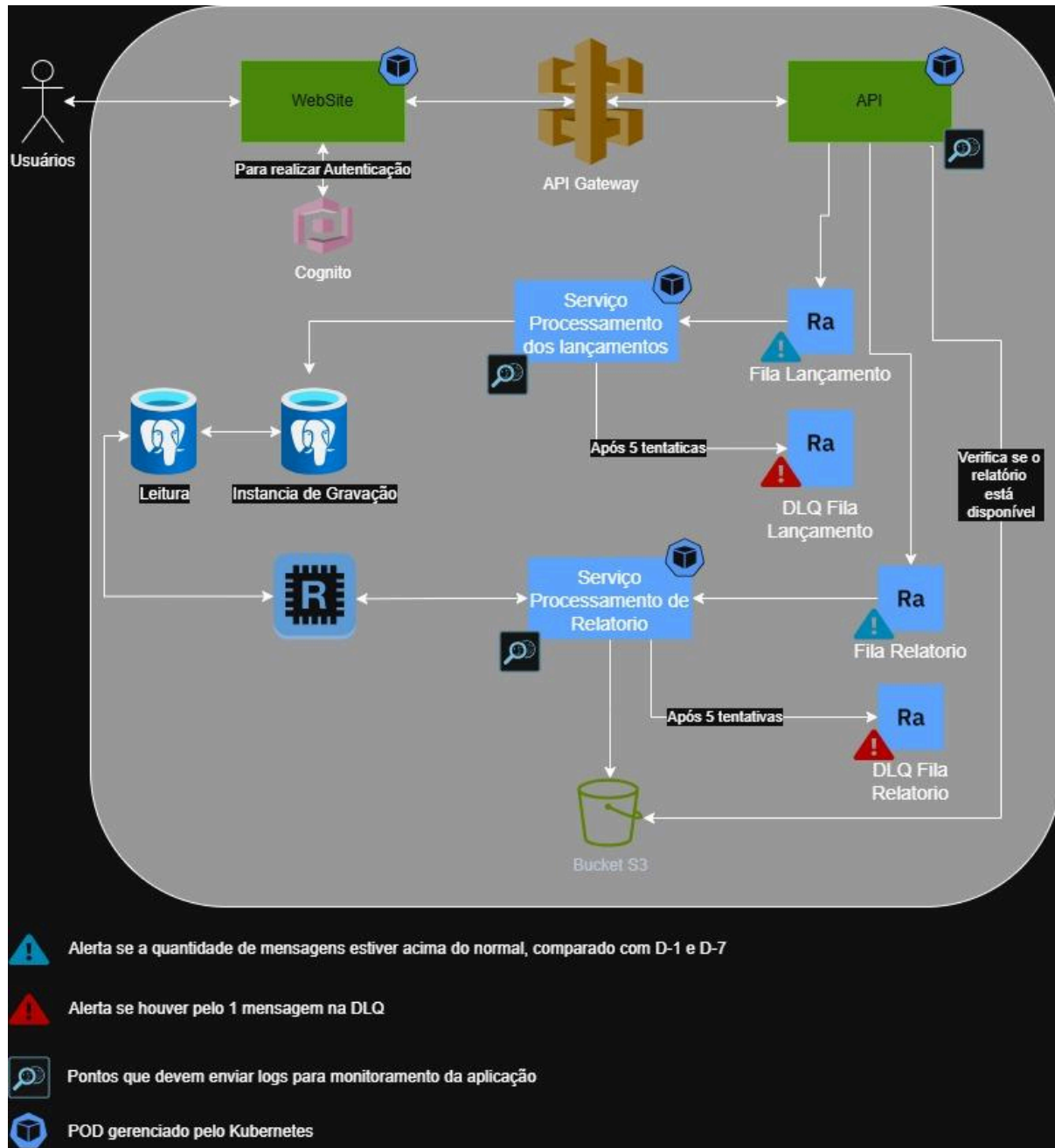


Desafio de arquitetura



1. Arquitetura

- **Microserviços:**

- Serviço de **Controle de Lançamentos**;
- Serviço de **Consolidado Diário**;

- A utilização de Microserviços, que é amplamente utilizado hoje por diversas empresas que buscam performance, escalabilidade e resiliência, se justifica neste caso por:
 - Permite escalar cada serviço de forma independente de acordo com sua demanda
 - Garante resiliência: um serviço não interfere no outro e se recupera de falhas em pouco tempo
 - Facilita evolução futura: o isolamento dos serviços e dos domínios irá garantir uma evolução mais simples e rápida.

2. Escalabilidade

- **Horizontal Scaling:** cada serviço pode ser replicado em múltiplas instâncias.
- **Kubernetes:** para garantir a escalabilidade do serviço de acordo com a demanda.
- **Cache:** uso de Redis para armazenar informações recorrentes e reduzir carga em picos.
- **Fila de Mensagens (RabbitMQ/Kafka):** Faz com que a estrutura seja assíncrona e resiliente a falhas. Utilizando filas, perder mensagens / dados e processamento se torna mais difícil.

3. Resiliência

- **Redundância:** Podemos ter pelo menos 2 pods para cada serviço, isso garantirá que se uma das instancias/pods parar, a outra irá assumir, garantindo que não perderemos tempo. A quantidade de instancias/pods será avaliada apos verificarmos o tempo de processamento de cada serviço.
- **Monitoramento:** Prometheus + Grafana ou DataDog para monitoramento dos serviços.

4. Segurança

- **Autenticação e Autorização:** JWT + OAuth2.
- **Criptografia:** HTTPS na comunicação entre os serviços e filas
- **Proteção contra ataques:** rate limiting, firewall de API (Azure API Management ou Kong).

5. Integração

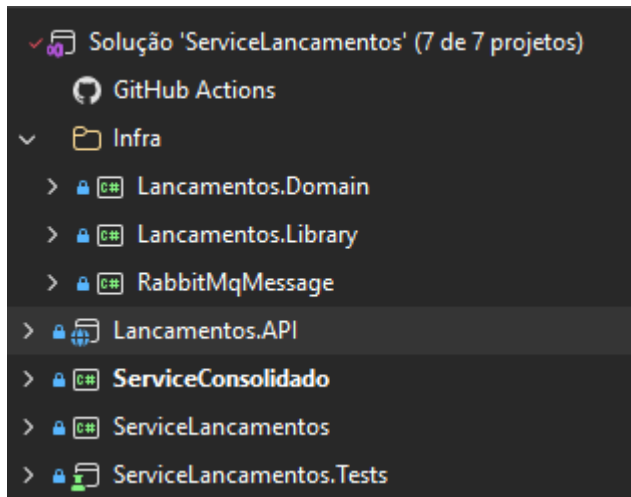
- **Comunicação assíncrona:** eventos via mensageria (Kafka/RabbitMQ).
- **Comunicação síncrona:** REST APIs com JSON.
- **Formato de Mensagem:** JSON padronizado, com versionamento de contratos.

6. Requisitos Não-Funcionais

- **Disponibilidade:** 99,9% para o serviço de lançamentos.
- **Desempenho:** consolidado diário deve suportar 50 req/s com até 5% de perda.
- **Confiabilidade:** logs centralizados

Stack Tecnológico:

- .NET 8 (Runtime moderno)
- Entity Framework Core 8 (ORM)
- RabbitMQ 7.2.1 (Message Broker)
- PostgreSQL 15 (Banco de Dados)
- Docker & Docker Compose (Containerização)



Lancamentos.Domain << contem todos os models utilizados

Lancamentos.Library << contem todo o tratamento com o banco de dados (entity Framework)

RabbitMqMessage << contem todo o tratamento e configuração do rabbit

Lancamentos.API << API que ficará após o API Gateway

ServiceConsolidado << Service responsável por gerar o Consolidado Diário

ServiceLancamentos << Service responsável por lançar os créditos e os débitos.

ServiceLancamentos.Tests << teste unitário utilizando xUnit

ATENÇÃO

1. Não foi implementado a segurança de **Autenticação e Autorização**: JWT + OAuth2, para não correr o risco de termos problemas no teste.
2. O Monitoramento, neste momento é feito através dos logs gerados.
3. Não foi implementado o Bucket na nuvem para disponibilizar os relatórios. No momento todos os relatórios são salvos em uma pasta, a ideia foi simular um bucket.

Arquitetura :

- Microservices + Event-driven (message-driven) — componentes desacoplados trocam mensagens via RabbitMQ.
- API Gateway (projecto Lancamento.API) expõe endpoints HTTP e publica mensagens.
- Worker Services (ServiceLancamentos, ServiceConsolidado) atuam como consumidores em background.
- Containerizado com Docker / docker-compose para orquestração.
- Separação em camadas: API (entrada), Domain/Library (modelo), Infrastructure (RabbitMqMessage), Services (processamento).

Padrões de projeto usados:

- Publisher–Subscriber / Message Broker (RabbitMQ) — desacoplamento via filas/exchanges.
- Repository (RabbitMqMessageRepository) — abstrai operações de envio/recebimento.
- Dependency Injection — configurável via Program.cs para injeção de conexões e serviços.
- BackgroundService / Hosted Service — para consumidores que rodam continuamente.
- Adapter / Ports-and-Adapters (RabbitMqDataContext como adaptador de infraestrutura).
- DTOs/Commands para mensagens (RabbitMQRequest, ConsolidadoRequest).
- (Em complemento) Strategy/Factory leve quando seleciona configurações de fila por tipo de mensagem.

A combinação voltada a alta escalabilidade e resiliência—API publica eventos e workers processam assincronamente, permitindo deploys e escalonamento independentes.